

Orthrus Field Manual

Everything you need to understand, defend, and debug a native Orthrus speculative-decoding backend for vLLM — with a runnable check at the end of every section.

RTX 5090 · sm_120 · WSL2 vLLM 0.28.1rc1.dev453+ga1541f574 September 2026

This manual exists because vLLM's `AGENTS.md` says “a human submitter must understand and defend the change end-to-end.” Every number in it was measured on your machine, not copied from a paper. Where a published figure and our measurement disagree, both appear and the discrepancy is explained.

CONTENTS

0 Orientation

I Why speculative decoding works at all

1. Decode is memory-bound
2. Verification is nearly free
3. The loop, and why it is exactly lossless

II What Orthrus does

1. Two views, one set of weights
2. One cycle, by hand
3. The reference loop

III Measuring it without fooling yourself

1. TPF and acceptance length differ by exactly 2×
2. Averaging convention is worth 8%
3. Sample size: ± 0.4 at $n=25$
4. What hardware can and cannot change
5. 80% of an eager pass is overhead

IV The six vLLM concepts you need

V Reading the proposer

1. The call order
2. The kernel, worked by hand
3. Why `seq_lens` subtracts `num_rejected`

VI **Building it**

A **Appendix: numbers and commands**

PART 0 Orientation

The environment

Two virtualenvs, deliberately separate.

PATH	HOLDS	WHY
<code>.venv</code>	vLLM, editable from <code>vllm/</code>	Our working copy. Edits take effect with no reinstall.
<code>.venv-ref</code>	torch + transformers only	The <i>numerical oracle</i> . A vLLM dependency bump must never silently change what we validate against.

That separation is not fussiness. Steps 1 and 2 of the implementation plan compare vLLM's output against the HuggingFace reference. If the reference lived in the same environment, a routine vLLM upgrade could move the oracle underneath you and you would never know.

TRAP — NAMESPACE SHADOWING

The repository root contains a directory called `vllm/`. Running `python -c "import vllm"` from the project root resolves to that *directory* as an empty namespace package, silently shadowing the install: `vllm.__file__` comes back `None`. Run from any other working directory. Script files are fine, because `sys.path[0]` is the script's own directory, not your shell's.

TRAP — WSL2 PINNED MEMORY

Every vLLM engine start fails with `RuntimeError: UVA is not available` unless you set `VLLM_WSL2_ENABLE_PIN_MEMORY=1`. This is not a bug: `vllm/platforms/cuda.py` disables pinned memory on WSL by default, and the v1 engine's `UvaBuffer` hard-requires it. Verified genuinely working here — pinned host-to-device runs 28.0 GB/s against 13.7 GB/s pageable, so it is real pinning and not a silent fallback.

How to use this manual

Every section ends with a **Verify** block: a command to run and the number it should produce. Run them. The single most important fact about this project is that a broken drafter produces *correct output* — so reading code until it “looks right” will not catch your bugs. Only measurement will.

PART I Why speculative decoding works at all

1.1 Decode is memory-bound

A transformer generating text runs in two regimes that look nothing alike to the hardware.

Prefill processes the whole prompt at once. Five hundred tokens go in, five hundred positions are computed in parallel. Big matrix multiplies. The GPU is compute-bound and happy.

Decode produces one token at a time, each depending on the last. To emit a single token you must stream *every weight in the model* from GPU memory and do a trivial amount of arithmetic with it. The GPU spends almost all its time waiting for weights to arrive.

Put your numbers to it. Orthrus-Qwen3-4B is 4.7B parameters in bfloat16 = 9.4 GB. Your RTX 5090 has about 1.79 TB/s of memory bandwidth:

```
9.4 GB / 1.79 TB/s = 5.3 ms    ← floor for ONE token
                                and the arithmetic is nearly free
```

Now the key observation: **processing 32 tokens in that same forward pass costs almost the same 5.3 ms**. You stream the weights once either way. That is the entire opportunity.

1.2 Verification is nearly free

Hand the model a sequence that is already filled in and one forward pass returns the next-token distribution at *every* position, because that is what prefill does. So:

- Generating K tokens with the target model: **K forward passes**.
- Checking K already-written tokens with the target model: **1 forward pass**.

If something cheap can propose plausible candidates, you check them in bulk and keep the ones that match.

1.3 The loop, and why it is exactly lossless

Take “The” and suppose we want to continue it.

```

STEP 1 – GUESS (one forward pass of the drafter)
in:  [ "The", MASK, MASK, MASK, MASK ]
out: [      "cat", "sat", "on", "the", "rug" ]

STEP 2 – CHECK (one forward pass of the real model)
in:  [ "The", "cat", "sat", "on", "the", "rug" ]
real:      "cat" "sat" "on" "the" "mat"

STEP 3 – COMPARE, left to right, stop at first mismatch
"cat" == "cat"    keep
"sat" == "sat"    keep
"on"  == "on"     keep
"the" == "the"    keep
"rug" != "mat"    STOP

keep 4 guesses + "mat" for free = 5 tokens for 2 passes

```

Everything after the first mismatch is discarded even if it happened to be correct. That rule is what makes the scheme exact: **every token emitted is a token the real model itself chose**, conditioned on exactly the tokens before it. The drafter cannot change *what* comes out, only *how fast*.

THE CONSEQUENCE THAT SHAPES THIS WHOLE PROJECT

A broken drafter never produces wrong text. It only produces slow text. Correctness lives entirely in the verification step, which is vLLM's existing, already-tested rejection sampler. So every bug you introduce will be invisible in the output and will show up *only* as a collapsed acceptance length. There is no crash, no wrong answer, no failing assertion – just a number that is 4 when it should be 7.8.

VERIFY

```
.venv-ref/bin/python bench/bench_eager.py --max-new-tokens 256
```

Expect three configurations and two invariants. `orthrus-diff` should be roughly twice `orthrus-ar`. Invariant I1 (Orthrus's AR view equals stock Qwen3) should pass 12/12. Invariant I2 will *fail* in bf16 – that is expected and explained in §3.4.

PART II What Orthrus does

2.1 Two views, one set of weights

Orthrus takes a frozen Qwen3 and adds, inside *every* transformer layer, a second set of attention projections. Everything else – embeddings, layer norms, MLPs, final norm, LM head – is shared and frozen.

VIEW	ROLE	CACHE BEHAVIOUR
AR	Ordinary Qwen3 forward. Prefill and verification.	Reads and writes as usual.
Diffusion	Takes K positions – anchor plus K–1 masks – and predicts all K–1 future tokens in one pass.	Reads the AR cache. Writes nothing permanent.

Verified against the checkpoint itself: the extra parameters are six tensors per layer – q_proj_diff, k_proj_diff, v_proj_diff, o_proj_diff, q_norm_diff, k_norm_diff – totalling **17.0%** of the model. They are full-rank nn.Linear; the r: 16 and lora_alpha: 32 fields in config.json are dead leftovers from training that nothing reads.

The diffusion view's queries attend to the AR cache *and* to the block's own keys and values, bidirectionally within the block. At inference there is no structured mask at all: every block query sees every context position and every block position. That is plain dense non-causal attention, which is why the FlashAttention backend is the right target and FlexAttention (which exists for arbitrary sparse masks) is not.

2.2 One cycle, by hand

The cache holds committed tokens up to position *t*–1.

```
positions:  0 ..... t-1 | t ..... t+K-1
            [cached] | (reserved) scheduler pre-allocated K slots

(a) DIFFUSION PASS  queries = the K-block
    reads  0 .. t-1  (valid target K/V)
    writes t .. t+K-1 (transient draft K/V)

(b) AR VERIFY PASS  over the K proposed tokens
    overwrites t .. t+K-1 with real target K/V

(c) CONSENSUS accepts 2
    cache is NOT cropped. seq_len is set to t+2.
    positions t+2 .. t+K-1 still hold real bytes – they are simply
    beyond seq_len and therefore invisible.
```

Two things follow, and both matter later. First, the slots the diffusion pass borrows are *exactly* the slots the verification pass is about to fill anyway – that is why Orthrus needs no extra cache memory. Second, rejected tokens are never erased; they are hidden by moving a length marker. Remember that.

2.3 The reference loop

Read `ref_peek/modeling_orthrus.py` lines 460–532 with this in front of you. The invariant the loop maintains is:

```
cache length == start_idx          and output_ids[start_idx] is the
                                   uncached anchor for the next block
```

Per cycle it builds `[anchor, MASK × (K-1)]` at positions `start_idx .. start_idx+K-1`, runs the diffusion pass with `use_cache=False` and `ar_seq_len=start_idx`, runs the AR pass with `use_cache=True`, compares, then calls `cache.crop(start_idx)`.

TRAP — THE REFERENCE IS BATCH-1 ONLY

`OrthrusLM.forward` accepts `attention_mask` but never passes it to `self.model`. Any padded batch silently attends to pad tokens. Fine as an oracle, useless for batched comparisons. Also:

`generation_config.json` sets `use_cache: false`, so AR generation must pass `use_cache=True` explicitly or every step recomputes the whole prefix.

VERIFY

```
.venv-ref/bin/python bench/compare_weights.py
```

Expect **310/310 tensors bit-identical** to stock Qwen3-1.7B, and the `*_diff` parameters at 17.0% of the total. The one apparent gap – Qwen3 storing a redundant tied `lm_head.weight` – equals Orthrus's `embed_tokens`, so it is not a real difference.

PART III Measuring it without fooling yourself

This part is longer than it looks like it should be. Every trap in it cost real time to find, and each one produces a plausible-looking wrong number rather than an error.

3.1 TPF and acceptance length differ by exactly 2×

TPF – tokens per forward pass, the Orthrus paper's Equation 9 and Table 1 – is generated tokens divided by *all* forward passes. Plain autoregressive decoding is TPF 1.0 by definition. An Orthrus cycle is two passes, so:

$$\text{TPF} = \text{tokens_per_cycle} / 2$$

Acceptance length – the Orthrus paper's Figure 4, the DFlash paper's Table 1, and vLLM's `vllm:spec_decode_*` metric – is accepted tokens per *verification* pass, which equals tokens per cycle, roughly $2 \times \text{TPF}$.

TRAP

Same run, two numbers, differing by exactly 2. Compare against Table 1 using TPF; compare against Figure 4 and vLLM's metric using tokens per cycle. Mixing them is a silent 2× error in either direction.

In vLLM the metric is defined in `tests/v1/e2e/spec_decode/utils.py` as `1 + accepted/drafts` – the `+1` is the bonus token the target samples after the last accepted draft.

3.2 Averaging convention is worth 8%

Two questions, invented for clarity:

```
Question A – short answer: 10 tokens, 2 passes → TPF 5.0
Question B – long answer: 90 tokens, 45 passes → TPF 2.0

equal-weighted (5.0 + 2.0) / 2 = 3.50
token-weighted 100 / 47      = 2.13
```

Same data. In the token-weighted version, question B supplies 90 of the 100 tokens and drowns out question A. And long answers systematically score lower, so the token-weighted mean always comes out lower.

Measured on 200 real GSM8K questions with Orthrus-Qwen3-1.7B:

CONVENTION	TPF	AGAINST PAPER'S 4.20
Equal-weighted (per question)	4.18	matches to 0.4%
Token-weighted (pooled)	3.88	8% low

The papers' figures line up with the equal-weighted convention. Report both or the comparison looks broken for no reason.

3.3 Sample size: ± 0.4 at $n=25$

Per-question TPF at 1.7B has mean 4.18 but **standard deviation 1.10**, ranging from 2.13 to 7.82. Some questions are easy to predict; some are not.

```
bootstrap, 4000 draws of 25 questions from our 200:  
95% CI = [3.52, 4.31]            $\pm 0.4$  wide
```

Our first 25-question run returned 3.78 against the paper's 4.20. That was never statistically distinguishable from agreement. **Use ≥ 100 questions** for any claim about matching a paper.

THE TELL

GSM8K came in *below* the paper; HumanEval came in *above*. A systematic cause – different hardware, a broken checkpoint – pushes both the same direction. Random scatter pushes them in random directions. When your errors change sign, suspect noise before you suspect a cause.

3.4 What hardware can and cannot change

TPF is a pure count. There is no time in the formula. Whether a guess is accepted is decided by `"cat" == "cat"` – arithmetic on fixed weights against fixed input. An H100 and an RTX 5090 compute the same argmax.

We proved this empirically rather than asserting it. DFlash on Qwen3-4B, the same 200 prompts, in two completely different frameworks:

HARNESS	ACCEPTANCE	LENGTH	TOK/S
HuggingFace eager		5.902	220.2
vLLM, CUDA graphs on		5.921	680.4

0.3% apart on acceptance; 3.1 $\times$ apart on throughput. Different kernels, different attention backend, graphs on versus off. That is the licence for everything else in this manual: acceptance length measures the *algorithm*, tokens per second measures the *implementation*.

The one way hardware leaks in is floating-point non-determinism, and it is small. Which brings us to:

TRAP — THE LOSSLESS TEST CANNOT PASS IN BF16

At temperature 0 the consensus rule is exact greedy prefix matching, so diffusion output *must* equal AR-only output. Measured: **fp32 12/12 identical, bf16 7/12**. Not a bug. Verification evaluates K=32 positions in one GEMM while the AR baseline does one position per GEMM; different reduction shapes round differently and flip the argmax at near-ties. The implementation guide states the Step 2 criterion as “byte-identical” in bf16 – taken literally, **that test fails on correct code**. Run the strict assertion in fp32; in bf16 assert a high match *rate*.

3.5 Eighty percent of an eager pass is overhead

Why is Orthrus-4B only 11% slower than Orthrus-1.7B when it has 2.3× the weights? Decompose the measured per-pass times against 1.79 TB/s:

MODEL	MS/PASS	WEIGHT STREAMING	REMAINDER	FITS
Orthrus-1.7B	15.5	4.15 GB → 2.3	13.2	28 layers × 0.47 ms
Orthrus-4B	22.2	9.40 GB → 5.3	16.9	36 layers × 0.47 ms

Both land on **~0.47 ms of overhead per layer**. In HuggingFace eager the model barely matters; the layer count does. Three consequences:

- The 4B looks cheap. It is not – the measurement is hiding its cost.
- A 5-layer drafter is not 7× cheaper in wall-clock, because it still pays per-layer overhead.
- **Eager systematically flatters both larger models and full-depth drafters – that is, it flatters Orthrus.** Expect rankings to move in vLLM.

TRAP — NEVER COMPARE TOK/S ACROSS TASKS

Orthrus-1.7B does 250.7 tok/s on GSM8K but 149.5 tok/s on a mixed prompt set containing prose. That is the prompt mix, not the model. Comparing the 1.7B's mixed-set number against the 4B's GSM8K number makes the larger model look faster, which it is not.

VERIFY

```
.venv-ref/bin/python bench/bench_eager.py --task gsm8k --num-prompts 200 \  
--max-new-tokens 512 --only orthrus-diff
```

Expect TPF **3.88** token-weighted, **4.18** per-question, acceptance length **7.75**, around 250 tok/s. Then run `--dtype fp32 --only orthrus-ar,orthrus-diff` and watch invariant I2 flip from 7/12 to 12/12.

PART IV The six vLLM concepts you need

1. Paged KV cache, block table, slot mapping

vLLM does not give each request a contiguous cache. Memory is divided into fixed-size **blocks** (16 tokens each) handed out on demand, like virtual memory pages. Each request has a **block table**: which physical blocks hold its tokens, in order. A **slot** is one token's position in one block, and the **slot mapping** says, for each input token, which physical slot its key and value get written to.

```
slot = block_table[req][position // 16] * 16 + (position % 16)
```

Everything about correctness in Part V comes down to computing that line right.

2. Scheduler

Decides each step which requests run and allocates their blocks. With speculative decoding on, it pre-allocates `num_speculative_tokens` of lookahead per request *before* the drafter runs – which is the only reason the diffusion pass has slots to borrow. For Orthus that is **31**, far above the 3–8 most drafters use; `vllm/config/speculative.py` documents DFlash as needing `K` additional slots.

3. Model runner

The GPU-side loop. Each step: build inputs, run the target, sample and verify, then call the drafter's `propose()` to produce drafts for the *next* step. That ordering is what makes Orthus safe – the transient draft K/V written at step n is overwritten by verification at step $n+1$, before anything else reads those slots.

4. Attention layer, backend, metadata

A vLLM `Attention` module owns no weights. It is a thin op that (i) looks up its KV-cache tensor *by layer name*, (ii) writes incoming k,v to the slots in the forward context, and (iii) calls a backend with the metadata there.

THIS IS THE CRUX OF THE WHOLE DESIGN

Calling the *same* `self.attn` module from both the AR path and the diffusion path means the same layer name, which means the same cache tensor – so the diffusion view gets the target's context K/V without copying a byte. Create a second `Attention` module per layer instead and vLLM registers it as a new attention layer, allocates it its own cache, and the entire memory argument for Orthus evaporates.

5. Forward context

A thread-local bag set for the duration of one forward, via `set_forward_context(...)`. It carries attention metadata and slot mappings, and any code inside the forward can read it. Drafters build their own for their own passes.

6. CUDA graphs

Launching hundreds of small kernels per decode step has overhead, so vLLM captures a step as a graph and replays it. Consequences for us: buffers the graph reads must live at **stable addresses**, and shapes must be padded to captured sizes. Get everything correct with `enforce_eager=True` first and add graph support last.

VERIFY

```
cd /tmp && VLLM_WSL2_ENABLE_PIN_MEMORY=1 \
/home/neukiru/workspace/orthrus_vllm/.venv/bin/python -c "
from vllm import LLM, SamplingParams
llm = LLM(model='Qwen/Qwen3-1.7B', max_model_len=1024, max_num_seqs=1,
          gpu_memory_utilization=0.25)
print(llm.generate(['The capital of France is'],
                  SamplingParams(temperature=0, max_tokens=8))[0].outputs[0].text)"
```

Expect “Paris. ...” and, in the log, a line reading `GPU KV cache size: N tokens` – that is the paged allocator reporting how many slots it carved out.

PART V Reading the proposer

5.1 The call order

The generic loop lives in `vllm/v1/spec_decode/llm_base_proposer.py` (1901 lines – you do not touch it). `vllm/v1/spec_decode/dflash.py` is 317 lines of overrides, and is the file our Orthrus proposer will most resemble.

```
propose(target_token_ids, target_positions, next_token_ids,
        cad, num_rejected_tokens_gpu, ...)
|
+ - set_inputs_first_pass(...)           ← the block + metadata [THE BIG ONE]
+ - build_per_group_and_layer_attn_metadata ← forces non-causal
+ - build_model_inputs_first_pass(...)
+ - with set_forward_context(...): self.model(**kwargs)
+ - sample_hidden_states = last_hidden_states[token_indices_to_sample]
+ - _sample_draft_tokens(...) → draft_token_ids
```

At `llm_base_proposer.py:627`, if `self.num_speculative_tokens == 1` or `self.parallel_drafting`: returns straight after one forward. Both DFlash and Orthrus set `parallel_drafting`, so there is a single draft pass and no loop – matching the Orthrus paper's Table 3 ablation, which found single-step projection beats multi-step (TPF 6.35 against 3.53).

5.2 The kernel, worked by hand

This is the checkpoint the implementation guide sets, so do it with real numbers – and with **two requests of different lengths**, because a single-request trace hides the two subtlest things in the kernel.

Take `num_speculative_tokens = 4`, so `num_query_per_req = 5`, and KV page size 16. Both requests were drafted for last step; the target has just finished verifying.

- **Request 0** had 10 committed tokens (positions 0–9). The target verified 5 query tokens at positions 10–14. Consensus accepted **2** drafts, so 3 tokens commit (10, 11, 12) and **2 are rejected** (13, 14).
- **Request 1** had 30 committed tokens (positions 0–29). The target verified 5 query tokens at positions 30–34. Consensus accepted **nothing**, so only 1 token commits (30) and **4 are rejected** (31–34).

```

inputs to copy_and_expand_dflash_inputs_kernel
query_start_loc      = [0, 5, 10]                request boundaries
target_positions     = [10,11,12,13,14, 30,31,32,33,34] flattened, ragged
num_rejected_tokens  = [2, 4]
next_token_ids       = [B0, B1]
cad.seq_lens         = [15, 35]
total_input_tokens   = 10
block_table_stride   = 8

block_table[0] = [42, 57, ...]    req 0: pos 0-15 → blk 42, 16-31 → blk 57
block_table[1] = [11, 23, 88, ...] req 1: 0-15 → 11, 16-31 → 23, 32-47 → 88

```

The launch grid

```

max_ctx_per_req      = cad.max_query_len          = 5
max_tokens_per_req   = 5 + 5                      = 10
BLOCK_SIZE           = min(256, next_power_of_2(10)) = 16
num_blocks           = ceil(10 / 16)              = 1
grid                 = (batch_size=2, 1)          two programs, one per request

```

Each program walks one request's context *and* query tokens under a single flat index `j`:

```

j:          0   1   2   3   4 | 5   6   7   8   9 | 10 .. 15
           └─ is_ctx (j < 5) ─┘ └─ is_query ─┘ └─ out of bounds ┘
query_off:          0   1   2   3   4

```

Request 0 — req_idx = 0

```

ctx_start = query_start_loc[0] = 0
ctx_end   = query_start_loc[1] = 5
num_ctx   = 5

the three pivotal lines, utils.py:520-526
num_rejected = 2
valid_ctx_end = ctx_end - num_rejected = 5 - 2 = 3
last_pos      = target_positions[3-1] = target_positions[2] = 12
query_pos     = 12 + 1 + query_off    = 13,14,15,16,17

```

It stepped *back over* positions 13 and 14 — the rejected ones — and resumed from 12.

```

context, j = 0..4
ctx_pos_idx = min(ctx_start + j, 9) = 0,1,2,3,4
ctx_pos      = target_positions[0..4] = 10,11,12,13,14
ctx_pos_out = ctx_start + j           = 0,1,2,3,4

query, j = 5..9 (query_off = 0..4)
query_out  = req_idx*5 + query_off = 0,1,2,3,4
query_pos  = 13,14,15,16,17

slot mapping: positions = where(is_ctx, ctx_pos, query_pos)
context pos 10 → 10//16=0 → blk 42 → 42*16+10 = 682
        pos 11 → 683      pos 12 → 684
        pos 13 → 685      pos 14 → 686
query   pos 13 → blk 42 → 685      ← same slot as context
        pos 14 → 686              ← same slot as context
        pos 15 → 687
        pos 16 → 16//16=1 → blk 57 → 57*16+0 = 912  ← PAGE BOUNDARY
        pos 17 → 913

input ids and sample indices
is_bonus      = is_query & (query_off == 0) → only j=5
out_input_ids[0..4] = [B0, M, M, M, M]
sample_out_idx = 0*4 + (query_off-1) = 0,1,2,3
value          = query_out           = 1,2,3,4

```

Request 1 — req_idx = 1

```

ctx_start = query_start_loc[1] = 5      not 0
ctx_end   = query_start_loc[2] = 10

num_rejected = 4
valid_ctx_end = 10 - 4 = 6
last_pos      = target_positions[6-1] = target_positions[5] = 30
query_pos     = 31,32,33,34,35

context, j = 0..4
ctx_pos_idx = min(5 + j, 9)             = 5,6,7,8,9
ctx_pos      = target_positions[5..9] = 30,31,32,33,34
ctx_pos_out = 5 + j                     = 5,6,7,8,9      offset by ctx_start

query, j = 5..9
query_out  = 1*5 + query_off           = 5,6,7,8,9      offset by req_idx*5
query_pos  = 31,32,33,34,35

slots — this request straddles two page boundaries
context pos 30 → 30//16=1 → blk 23 → 23*16+14 = 382
        pos 31 → 383
        pos 32 → 32//16=2 → blk 88 → 88*16+0 = 1408
        pos 33 → 1409      pos 34 → 1410
query   pos 31 → 383      pos 32 → 1408      pos 33 → 1409
        pos 34 → 1410      pos 35 → 1411

```


The two things one request would have hidden

1. The context and query buffers use different indexing schemes.

```
context:  ctx_pos_out = ctx_start + j          ragged, follows the target's flat layout
query:    query_out   = req_idx * 5 + query_off uniform, fixed stride
```

Context is ragged because requests contribute different numbers of tokens to the target's pass. Queries are uniform because *every* request always gets exactly `1 + num_speculative_tokens` of them. That is why `new_query_start_loc` is simply `arange * num_query_per_req` – regular by construction.

2. `last_pos` is read at a per-request offset. For request 1 it is `target_positions[5]`, not `[0].valid_ctx_end` indexes the *flattened* array, which is why it is computed from `ctx_end` and not from `num_ctx`.

Final buffer contents

INDEX	0	1	2	3	4	5	6	7	8	9
<code>out_context_positions</code>	10	11	12	13	14	30	31	32	33	34
<code>out_context_slot_mapping</code>	682	683	684	685	686	382	383	1408	1409	1410
<code>out_query_positions</code>	13	14	15	16	17	31	32	33	34	35
<code>out_query_slot_mapping</code>	685	686	687	912	913	383	1408	1409	1410	1411
<code>out_input_ids</code>	B0	M	M	M	M	B1	M	M	M	M
<code>out_token_indices</code> (len 8)	1	2	3	4	6	7	8	9		

Two details worth pausing on. **The slot mapping crosses page boundaries by itself** – request 0's positions 16–17 land in a different physical block, request 1's cross twice – so the kernel does a real block-table lookup per position rather than assuming contiguity. Do not “simplify” it; if a computed slot ever falls outside the request's block table you silently corrupt another request's cache.

And **query slots overlap context slots**: 685 and 686 appear in both of request 0's lists. Those positions hold rejected K/V; the query pass overwrites them with the anchor's real K/V. Self-correcting, and the reason the kernel does not bother branching around rejected tokens when writing context.

5.3 Why `seq_lens` subtracts `num_rejected`

Continuing the same two-request example:

```

num_query_total      = 2 * 5 = 10
new_query_start_loc = arange[0..2] * 5 = [0, 5, 10]

effective_seq_lens   = cad.seq_lens - num_rejected_tokens_gpu
                    = [15, 35] - [2, 4]   = [13, 31]
new_cad.seq_lens     = [13, 31] + 5       = [18, 36]

check both
req 0:  13 context (0..12) + 5 query (13..17) = 18  ✓
req 1:  31 context (0..30) + 5 query (31..35) = 36  ✓

```

Recall from §2.2: **vLLM never erases rejected tokens**. They sit in the cache holding real, target-computed K/V. What makes them “gone” is purely that `seq_len` no longer reaches them.

TRAP — THE SINGLE MOST LIKELY CAUSE OF A FAILED LOSSLESS TEST

Forget the subtraction and you get:

```

req 0:  15 + 5 = 20   → context becomes 0..14, including rejected 13,14
req 1:  35 + 5 = 40   → context becomes 0..34, including rejected 31,32,33,34

```

Request 1 is the vivid case: it accepted *nothing*, so four of its five context tokens are garbage. **The drafter would condition on tokens the target explicitly threw away.** And per §1.3, the output stays perfect — verification catches every bad draft. You would see acceptance length quietly fall from 7.8 to around 4, and a “why is this slower than the paper” that looks like a hardware or configuration problem. This is exactly why the target numbers in §3 were pinned down before any code was written.

VERIFY — ANSWER THESE FROM MEMORY

1. What does each of the six output buffers hold, and why does `out_token_indices_to_sample` start at 1 rather than 0?
2. Why can a request's query slots be non-contiguous?
3. If you deleted the `- num_rejected`, what would the *test output* look like — and what would the metric do?

If you can answer these you have cleared the guide's Step 0 checkpoint.

PART VI Building it

6.1 The diff against DFlash

Orthrus's config gives `block_size = 32`, so `num_query_per_req = 32 = 1 anchor + 31 masks`. The kernel's `[bonus, MASK×n]` layout is Orthrus's `[anchor, MASK×(K-1)]`: the guide's glossary confirms the bonus token is the anchor.

IN DFLASH.PY	FOR ORTHRUS
<code>pass_hidden_states_to_model=True</code>	<code>False</code> — we feed token ids
<code>precompute_and_store_context_kv(...)</code> , 2 sites	delete — context K/V is already in the target's cache
<code>_context_positions_buffer</code> , <code>_context_slot_mapping_buffer</code>	delete
<code>combine_hidden_states</code> path	not applicable
<code>_get_model()</code> → separate draft model	return <code>self.runner.model</code>
draft KV cache registered	none — <code>draft_attn_layer_names</code> becomes empty
<code>set_inputs_first_pass</code>	keep nearly verbatim, including - <code>num_rejected</code>
non-causal metadata assertion	keep verbatim

Our file should be *shorter* than DFlash's 317 lines, because every deletion removes an entire subsystem.

DRIFT FROM THE IMPLEMENTATION GUIDE

The guide says to auto-detect Orthrus via `model_type == "orthrus"`. The real config says `"model_type": "qwen3"` with `"architectures": ["OrthrusLM"]`. Detection must key on the **architecture name**.

6.2 Steps, each with an acceptance test

- 1. **Model class.** `OrthrusQwen3ForCausalLM` with weight loading; the three `*_proj_diff` fuse into `qkv_proj_diff` exactly as Qwen3 fuses q/k/v. *Test:* ordinary generation matches stock Qwen3-1.7B, then diffusion-pass logits match the HF reference to bf16 tolerance.

- 2. **Proposer, batch 1, greedy, eager.** *Test:* the numbers in §6.3.
- 3. **Batch > 1, then sampling.** Per-request `seq_lens` and `query_start_loc` must be right with mixed prompt lengths.
- 4. **CUDA graphs and performance.** Turn off `enforce_eager` ; captured shape is `B_pad × 32` query tokens, full depth.
- 5. **Upstream.** RFC first – see §6.4.

6.3 Acceptance criteria

MEASUREMENT	TARGET
GSM8K TPF, 1.7B, token-weighted, n≥100	≈ 3.9
GSM8K acceptance length, 1.7B	≈ 7.8
GSM8K TPF, 4B	≈ 5.0
Lossless, fp32	100% identical
Lossless, bf16	high match rate, near-ties only

Within a few percent is kernel numerics. **A drop beyond ~5% is a bug** – and look at `num_rejected` first.

6.4 How to pitch it

Measured head-to-head against DFlash on Qwen3-4B, same harness, same 200 GSM8K questions:

HARNESS	METHOD	TOK/S	TOK/CYCLE
HF eager	Orthrus-4B	223.7	9.95
HF eager	DFlash-4B	220.2	5.90
vLLM	AR baseline	155.7	–
vLLM	DFlash-4B	680.4	5.92
vLLM	Orthrus-4B (<i>projected</i>)	~696	9.95

Roughly a tie. Orthrus drafts far better – 9.95 tokens per cycle against 5.90 – but pays for it with a full-depth draft pass where DFlash pays a fifth of one. Measured directly: a DFlash draft pass is 3.20 ms against a 21.53 ms target pass, a ratio of 0.149.

DO NOT PITCH ORTHRUS ON THROUGHPUT

The defensible case is **zero extra KV cache** (DFlash carries a second cache, ~14% of the target's, growing per request with context length), one checkpoint instead of two, and no separate drafter to train or ship. A speed claim may not survive review; the memory claim will. And note the tension: Orthrus's memory advantage only pays at high concurrency, but its two full-depth passes hurt most there too. Nobody has measured which wins – that is the most valuable experiment still open.

VERIFY

```
.venv-ref/bin/python bench/bench_dflash.py --task gsm8k --num-prompts 200 --size 4b
.venv-ref/bin/python bench/measure_pass_cost.py --num-prompts 10
.venv/bin/python bench/bench_vllm.py --task gsm8k --num-prompts 200 --size 4b
```

APPENDIX Numbers and commands

A.1 Every number worth remembering

QUANTITY	VALUE	SOURCE
Orthrus block size K	32	<code>config.json</code>
<code>num_speculative_tokens</code>	31	$K - 1$
<code>mask_token_id</code>	151669	<code>config.json</code>
New parameters	17.0%	measured; guide says ~16%
AR tensors identical to Qwen3	310 / 310	<code>compare_weights.py</code>
Per-question TPF sd (1.7B)	1.10	n=200
95% CI at n=25	± 0.4	bootstrap
Eager overhead per layer	0.47 ms	decomposition, §3.5
DFlash draft / target pass	0.149	direct timing
RTX 5090 bandwidth	~1.79 TB/s	spec

A.2 Commands

```
# baseline + both invariants
.venv-ref/bin/python bench/bench_eager.py --max-new-tokens 256

# paper comparison (use >=100 prompts)
.venv-ref/bin/python bench/bench_eager.py --task gsm8k --num-prompts 200 \
    --max-new-tokens 512 --only orthrus-diff

# 4B
.venv-ref/bin/python bench/bench_eager.py --task gsm8k --num-prompts 200 \
    --max-new-tokens 512 --only orthrus-diff --orthrus-model chiennv/Orthrus-Qwen3-4B

# strict lossless test – must be fp32
.venv-ref/bin/python bench/bench_eager.py --dtype fp32 --only orthrus-ar,orthrus-diff

# weights really frozen?
.venv-ref/bin/python bench/compare_weights.py

# DFlash, same harness
.venv-ref/bin/python bench/bench_dflash.py --task gsm8k --num-prompts 200 --size 4b
.venv-ref/bin/python bench/measure_pass_cost.py --num-prompts 10

# vLLM side (env var is set automatically by the script)
.venv/bin/python bench/bench_vllm.py --task gsm8k --num-prompts 200 --size 4b
```

A.3 Reading order

1. `ref_peek/modeling_orthrus.py` – lines 107–182 (attention), 460–532 (loop)
2. `vllm/v1/spec_decode/dflash.py` – all 317 lines
3. `vllm/v1/spec_decode/utils.py:458` – the kernel
4. `vllm/model_executor/models/qwen3.py` – attention and `load_weights`
5. `vllm/v1/spec_decode/llm_base_proposer.py` – `__init__`, `propose`, `load_model`, `_get_model`, `set_inputs_first_pass`. Skip the rest.

Every figure measured on this machine, September 2026. Where a published number and a measured one disagree, both are shown. vLLM internals move quickly – if a method name here has changed, the role it plays will still exist; grep for the DFlash proposer and follow what it does.